

# K-KAN: Complete Mathematical Reference

Kernels · KAN Layers · Variational & Meshless PDE Solvers · Derivative Learning  
· Mixed-Formulation 4th-Order PDEs

Covers every module in the codebase, in build order

## Contents

|     |                                                                                               |    |
|-----|-----------------------------------------------------------------------------------------------|----|
| 1   | Overview and Notation .....                                                                   | 2  |
| 2   | Kernel Families ( <code>kkan/kernels/families.py</code> ) .....                               | 3  |
| 2.1 | Positive Definiteness .....                                                                   | 3  |
| 2.2 | Implemented Families .....                                                                    | 3  |
| 2.3 | Polynomial Augmentation ( <code>PolyAugKernel</code> ) .....                                  | 3  |
| 3   | Kernel Derivatives ( <code>kkan/kernels/derivatives.py</code> ) .....                         | 4  |
| 3.1 | Analytic Derivative Formulas .....                                                            | 4  |
| 3.2 | Safety at $r = 0$ .....                                                                       | 4  |
| 4   | K-KAN Layer ( <code>kkan/model/kan_layer.py</code> ) .....                                    | 5  |
| 4.1 | Adaptive Centre Placement .....                                                               | 5  |
| 4.2 | Moving-Least-Squares (MLS) Normalisation .....                                                | 5  |
| 4.3 | Edge Activation .....                                                                         | 5  |
| 5   | Hard-Concrete Pruning ( <code>kkan/model/kan_layer.py, pruning.py</code> ) .....              | 6  |
| 6   | Variational PDE Solvers ( <code>kkan/solvers/variational_solver.py</code> ) .....             | 7  |
| 6.1 | Deep Ritz (Energy Minimisation) .....                                                         | 7  |
| 6.2 | Least-Squares Petrov-Galerkin (LSPG) .....                                                    | 7  |
| 6.3 | Adaptive Collocation .....                                                                    | 7  |
| 6.4 | XPINN Domain Decomposition .....                                                              | 7  |
| 7   | Direct Meshless Solvers ( <code>kkan/solvers/meshless_solver.py</code> ) .....                | 8  |
| 7.1 | Kansa Collocation .....                                                                       | 8  |
| 7.2 | Symmetric Galerkin .....                                                                      | 8  |
| 7.3 | RKPM / MLS Interpolant .....                                                                  | 8  |
| 7.4 | Method of Lines .....                                                                         | 8  |
| 8   | Derivative and Gradient Learning .....                                                        | 9  |
| 8.1 | Hermite-Birkhoff Collocation ( <code>kkan/approximation/hermite_collocation.py</code> ) ..... | 9  |
| 8.2 | Super-Convergence Theorem .....                                                               | 9  |
| 8.3 | Regularisation: Tikhonov vs SVD Truncation .....                                              | 10 |
| 8.4 | Gradient-Enhanced K-KAN Edges ( <code>kkan/model/gradient_enhanced_layer.py</code> ) .....    | 10 |
| 8.5 | High-Level API ( <code>kkan/model/joint_training.py</code> ) .....                            | 11 |
| 9   | Mixed-Formulation 4th-Order PDEs ( <code>kkan/solvers/coupled_solver.py</code> ) .....        | 12 |
| 9.1 | Why a Single Scalar Network Fails for 4th-Order PDEs .....                                    | 12 |
| 9.2 | The Mixed-Formulation Fix .....                                                               | 12 |
| 9.3 | Boundary Conditions for the Cavity Flow Example .....                                         | 12 |
| 10  | Summary Table: Module -> Mathematics Map .....                                                | 13 |
| 11  | References .....                                                                              | 13 |

# 1 Overview and Notation

Throughout,  $\mathbf{x} \in \mathbb{R}^d$  denotes a spatial (or space-time) point,  $\mathbf{c}_k$  a kernel centre,  $\sigma$  a bandwidth, and  $r = \|\mathbf{x} - \mathbf{c}\|/\sigma$  the normalised radial distance used by every kernel. A K-KAN layer maps  $\mathbb{R}^{d_\ell} \rightarrow \mathbb{R}^{d_{\ell+1}}$  by summing, for each output node  $j$ , a **learned univariate function** of each input coordinate  $x_i$ :

$$x_j^{(\ell+1)} = \sum_{i=1}^{d_\ell} \varphi_{ij}^{(\ell)}(x_i^{(\ell)}).$$

Each edge function  $\varphi_{ij}$  is represented as a kernel expansion — this single design choice is what connects the whole codebase to classical meshless/RBF approximation theory, and is why every PDE solver, derivative estimator, and pruning mechanism below is expressed in kernel language.

## 2 Kernel Families (`kkan/kernels/families.py`)

### 2.1 Positive Definiteness

A radial kernel  $k(\mathbf{x}, \mathbf{c}) = \varphi(\|\mathbf{x} - \mathbf{c}\|)$  is **positive definite** on  $\mathbb{R}^d$  iff for all finite point sets and coefficients  $\alpha \neq 0$ ,

$$\sum_{i,j} \alpha_i \alpha_j \varphi(\|\mathbf{x}_i - \mathbf{x}_j\|) > 0.$$

Equivalently (Bochner),  $\varphi$  is the Fourier transform of a positive measure. This guarantees kernel (Gram) matrices are symmetric positive-definite for distinct sites, so interpolation systems are solvable.

### 2.2 Implemented Families

| Kernel                | Formula $k(r)$                                   | Smooth.    | Support          |
|-----------------------|--------------------------------------------------|------------|------------------|
| Gaussian RBF          | $\exp(-r^2)$                                     | $C^\infty$ | Global           |
| Inverse Multiquadric  | $(1 + r^2)^{-\beta}$                             | $C^\infty$ | Global           |
| Matérn- $\frac{1}{2}$ | $e^{-r}$                                         | $C^0$      | Global           |
| Matérn- $\frac{3}{2}$ | $(1 + \sqrt{3}r)e^{-\sqrt{3}r}$                  | $C^2$      | Global           |
| Matérn- $\frac{5}{2}$ | $(1 + \sqrt{5}r + 5\frac{r^2}{3})e^{-\sqrt{5}r}$ | $C^4$      | Global           |
| Wendland $C^{2k}$     | piecewise polynomial $\times (1 - r)_+^l$        | $C^{2k}$   | Compact, $r < 1$ |
| Wu $C^{2k}$           | piecewise polynomial $\times (1 - r)_+^l$        | $C^{2k}$   | Compact, $r < 1$ |

Table 1: Kernel families, `kkan/kernels/families.py`.

The native-space (Sobolev) order of each kernel — the exponent  $\tau$  governing interpolation convergence  $\|u - s_h\|_{L^\infty} = O(h^\tau)$  — is: Matérn- $\nu$ :  $\tau = \nu + \frac{d}{2}$ ; Wendland- $C^{2k}$ :  $\tau = k + \frac{d}{2} + \frac{1}{2}$ ; RBF:  $\tau = \infty$ .

### 2.3 Polynomial Augmentation (`PolyAugKernel`)

For compactly-supported kernels, the raw kernel matrix can be singular when scattered points fall outside each other's support. Augmenting with a polynomial tail restores unisolvence:

$$s(\mathbf{x}) = \sum_j w_j k(\mathbf{x}, \mathbf{x}_j) + \sum_{|\alpha| \leq m} q_\alpha \mathbf{x}^\alpha,$$

with the **annihilation condition**  $\sum_j w_j \mathbf{x}_j^\alpha = 0$  for all  $|\alpha| \leq m$ , giving the augmented saddle-point system

$$\begin{pmatrix} K & P \\ P^\top & 0 \end{pmatrix} \begin{pmatrix} \mathbf{w} \\ \mathbf{q} \end{pmatrix} = \begin{pmatrix} \mathbf{f} \\ 0 \end{pmatrix}.$$

The Legendre basis (`PolyAugKernel.legendre`, Bonnet recursion  $P_n = ((2n - 1)xP_{n-1} - (n - 1)P_{n-2})/n$ ) is used instead of raw monomials because it is orthogonal on  $[-1, 1]$ , avoiding the Vandermonde-matrix ill-conditioning that plagues monomial bases beyond degree  $\sim 6$ .

### 3 Kernel Derivatives (`kkan/kernels/derivatives.py`)

#### 3.1 Analytic Derivative Formulas

With  $r = \|\mathbf{x} - \mathbf{c}\|/\sigma$ , the chain rule gives:

$$\frac{\partial k}{\partial x_i} = k'(r) \cdot \frac{x_i - c_i}{\sigma^2 r}, \quad \frac{\partial^2 k}{\partial x_i \partial x_j} = \frac{k''(r)}{\sigma^4} (x_i - c_i) \frac{x_j - c_j}{r^2} + \frac{k'(r)}{\sigma^2} \left( \frac{\delta_{ij}}{r} - (x_i - c_i) \frac{x_j - c_j}{\sigma^2 r^3} \right),$$

$$\Delta_{\mathbf{x}} k = \text{tr}(\nabla_{\mathbf{x}}^2 k) = (k''(r) + (d-1)k'(r)/r)/\sigma^2.$$

**Correctness note:** an earlier version of this code had two independent scaling/sign bugs in these formulas — a missing  $\frac{1}{\sigma}$  factor in  $\partial_{\delta}^r x_i$ , and an algebraically incorrect Hessian assembly. Both were traced by comparing against `jax.grad/jax.hessian` autodiff ground truth on all four kernel families and are now fixed; the formulas above are the verified, correct closed forms.

#### 3.2 Safety at $r = 0$

Since kernel centres frequently coincide with evaluation points (e.g. the diagonal of a collocation matrix, where  $\mathbf{x}_i = \mathbf{c}_i$ ), and  $\|\cdot\|$  has an undefined second derivative at the origin, naively differentiating  $r$  via `jax.grad/jax.hessian` through `jnp.linalg.norm` produces **NaN** on the diagonal. The fix used throughout is a safe floor  $r \leftarrow \max(r, \varepsilon)$ ,  $\varepsilon = 10^{-7}$ , applied **before** evaluating  $k'(r)$ ,  $k''(r)$  — since  $k'(0) = 0$  for every kernel here, the floored value introduces negligible bias while eliminating the division-by-zero.

## 4 K-KAN Layer (`kkan/model/kan_layer.py`)

### 4.1 Adaptive Centre Placement

Centres are initialised at Gauss-Legendre or Chebyshev quadrature nodes rather than a uniform grid:

$$c_k = \frac{(\xi_k + 1)}{2}(b - a) + a, \quad P_K(\xi_k) = 0 \text{ (Gauss); } \quad c_k = \cos(\pi k / (K - 1)) \text{ (Chebyshev)}.$$

Both node families minimise the Lebesgue constant  $\Lambda_K = \max_x \sum_k |\ell_{k(x)}|$  for polynomial-type interpolation — Chebyshev achieves  $\Lambda_K = O(\log K)$  vs  $O(2^K / (K \log K))$  for uniform spacing, directly suppressing Runge-type oscillation.

### 4.2 Moving-Least-Squares (MLS) Normalisation

Raw kernel activations  $\varphi_{k(\mathbf{x})} = k(r_k)$  are optionally renormalised:

$$\tilde{\varphi}_{k(\mathbf{x})} = \varphi_{k(\mathbf{x})} / \left( \sum_m \varphi_{m(\mathbf{x})} + \varepsilon \right).$$

This enforces the **partition-of-unity** property  $\sum_k \tilde{\varphi}_k = 1$ , guaranteeing exact reproduction of constants (the zeroth Strang-Fix condition) without a bias term, and bounding the kernel contribution to  $[\min_k w_k, \max_k w_k]$  regardless of  $\sigma$ .

**Trade-off:** for compactly-supported kernels, normalisation destroys exact sparsity (any nonzero denominator term makes all numerators nonzero), so it should be disabled (`use_mls_norm=False`) whenever sparse kernel matrices matter (e.g. the meshless MOL solver).

### 4.3 Edge Activation

$$\varphi_{ij}(x_i) = \sum_{k=1}^K w_{ijk} \tilde{\varphi}_k(x_i) + v_{ij} \text{SiLU}(x_i) + \sum_{\beta} q_{ij\beta} P_{\beta}(x_i).$$

The full layer output, vectorised over batch  $B$ :

$$y_{bj} = \sum_{i=1}^{d_\ell} \text{gate}_{ij} \cdot \left[ \sum_k w_{ijk} \tilde{\varphi}_{bik} + v_{ij} \text{SiLU}(x_{bi}) + \dots \right].$$

Kernels are elementwise functions of the pre-computed distance tensor  $\mathbf{r}$  of shape  $(B, \text{in}, \text{out}, K)$ ; they are applied **directly** rather than via nested `jax.vmap`. (An earlier implementation wrapped a 4-fold nested `vmap` around each already-elementwise kernel call — mathematically a no-op, but it inflated JAX’s tracing graph so severely that even trivial models took tens of seconds to JIT-compile; removing the redundant `vmap` layers had zero effect on outputs and a  $> 10 \times$  speedup on compile time.)

## 5 Hard-Concrete Pruning (`kkan/model/kan_layer.py`, `pruning.py`)

Each edge  $(i, j)$  carries a gate

$$g_{ij} = \text{clip}(\sigma(\log \alpha_{ij})(\zeta - \gamma) + \gamma, 0, 1),$$

with stretch parameters  $\gamma = -0.1$ ,  $\zeta = 1.1$  ensuring  $g$  can reach exactly 0 or 1. The differentiable expected  $L_0$  penalty is

$$\mathbb{E}[\|g\|_0] = \sum_{i,j} \sigma(\log \alpha_{ij} - \beta \log(-\gamma/\zeta)),$$

added to the training loss with a cubic warm-up schedule

$$\lambda_0(t) = \lambda_{\max} \min(1, ((t - t_w)/(T - t_w))^3)$$

so pruning pressure only activates after the network has had time to find a good dense solution.

**Compute cost:** enabling pruning requires computing both the gated and (implicitly, via the gate’s gradient) ungated contribution per edge, roughly doubling forward/backward cost. Combined with deep derivative graphs (Hessians, bi-Laplacians), this compounding cost is severe enough that pruning is disabled by default for all 4th-order PDE examples (see §9) to avoid XLA compiler instability.

## 6 Variational PDE Solvers (`kkan/solvers/variational_solver.py`)

### 6.1 Deep Ritz (Energy Minimisation)

For the self-adjoint elliptic problem  $-\nabla a \nabla u = f$ ,  $u = g$  on  $\partial\Omega$ , the energy functional

$$\mathcal{E}[u] = \frac{1}{2} \int_{\Omega} a \|\nabla u\|^2 dx - \int_{\Omega} f u dx$$

is minimised by the unique weak solution (Euler-Lagrange = the PDE). Discretised with Gauss quadrature  $(\mathbf{x}_i, w_i)$ :

$$\mathcal{E}[u_{\theta}] \approx \frac{1}{2} \sum_i w_i a(\mathbf{x}_i) \|\nabla u_{\theta}(\mathbf{x}_i)\|^2 - \sum_i w_i f(\mathbf{x}_i) u_{\theta}(\mathbf{x}_i) + \beta_{bc} \frac{1}{N_b} \sum_j \left( u_{\theta}(\mathbf{x}_j) - g(\mathbf{x}_j) \right)^2.$$

Because  $\mathcal{E}$  is strictly convex in  $u$  (for  $a \geq a_0 > 0$ ), this loss landscape is far better-conditioned than the strong-form residual — no second derivatives of  $u_{\theta}$  are needed (only  $\nabla u_{\theta}$ ).

### 6.2 Least-Squares Petrov-Galerkin (LSPG)

For non-self-adjoint or time-dependent PDEs, minimise

$$\mathcal{J}[u_{\theta}] = \frac{1}{2} \sum_i w_i |\mathcal{L}[u_{\theta}](\mathbf{x}_i)|^2 + \beta_{bc} \|u_{\theta} - g\|_{\partial\Omega}^2 + \beta_{ic} \|u_{\theta} - u_0\|_{t=0}^2.$$

Collocation points use Gauss (tensor-product,  $n^d$  points) or Halton (low-discrepancy, exactly  $n$  points, no curse of dimensionality) quadrature.

### 6.3 Adaptive Collocation

Residual-based resampling draws new points  $\propto |\mathcal{L}[u_{\theta}]|^2$ , concentrating effort where the PDE is hardest to satisfy (shocks, boundary layers) — the standard RAR/RUCA strategy.

### 6.4 XPINN Domain Decomposition

$\Omega = \bigcup_k \Omega_k$ , each sub-domain trained independently with interface value-continuity enforced as an alternating (Gauss-Seidel-like) soft boundary condition using the neighbour's current prediction.

## 7 Direct Meshless Solvers (`kkan/solvers/meshless_solver.py`)

### 7.1 Kansa Collocation

Ansatz  $u(\mathbf{x}) = \sum_j w_j k(\mathbf{x}, \mathbf{x}_j) + \text{poly tail}$ ; enforce the PDE **pointwise** at interior points and the BC at boundary points:

$$\begin{pmatrix} A_{\text{int}} & P_{\text{int}} \\ A_{\text{bc}} & P_{\text{bc}} \\ P_{\text{c}}^\top & 0 \end{pmatrix} \begin{pmatrix} \mathbf{w} \\ \mathbf{q} \end{pmatrix} = \begin{pmatrix} -\mathbf{f} \\ \mathbf{g} \\ 0 \end{pmatrix},$$

with  $A_{\text{int}}[i, j] = -\Delta_{\mathbf{x}} k(\mathbf{x}_i, \mathbf{x}_j)$  computed via the **analytic** (now-corrected, NaN-safe) Laplacian formula of §4 — critically important since  $\mathbf{x}_i = \mathbf{c}_j$  occurs on the diagonal.

### 7.2 Symmetric Galerkin

Forms normal equations  $B^\top B + \lambda_{\text{bc}} B_{\text{bc}}^\top B_{\text{bc}}$ , symmetric positive (semi-)definite by construction, trading a squared condition number ( $\kappa(B^\top B) = \kappa(B)^2$ ) for guaranteed SPD structure amenable to Cholesky.

### 7.3 RKPM / MLS Interpolant

At each query point, solves a small local system

$$(P^\top W(\mathbf{x}) P) \mathbf{a}(\mathbf{x}) = P^\top W(\mathbf{x}) \mathbf{u}, \quad \hat{u}(\mathbf{x}) = P_b(\mathbf{x})^\top \mathbf{a}(\mathbf{x}),$$

$W(\mathbf{x}) = \text{diag}(k(\mathbf{x} - \mathbf{x}_j))$  — reproduces polynomials of degree  $\leq m$  exactly regardless of bandwidth.

### 7.4 Method of Lines

Backward-Euler time-stepping for  $u_t = \kappa \Delta u + f$ :

$$(K_{\text{int}} - dt, \kappa, L_{\text{int}}) \mathbf{w}^{n+1} = K_{\text{int}} \mathbf{w}^n + dt, \mathbf{f}^{n+1},$$

unconditionally stable ( $A$ -stable) but only  $O(dt)$  accurate.

## 8 Derivative and Gradient Learning

This section documents the joint value+gradient approximation machinery and, in detail, a subtle sign-convention bug that silently corrupted gradient accuracy in earlier versions — its diagnosis and fix are instructive for anyone extending the Hermite collocation code.

### 8.1 Hermite-Birkhoff Collocation (`kkan/approximation/hermite_collocation.py`)

**Ansatz.** Both the value-basis and derivative-basis functions are functions of the **free evaluation variable**  $\mathbf{x}$ , with the data site  $\mathbf{x}_j$  held fixed as a parameter:

$$s(\mathbf{x}) = \sum_j w_j k(\mathbf{x}, \mathbf{x}_j) + \sum_j \sum_l v_{j,l} \partial_l [k(\mathbf{x}, \mathbf{x}_j)] + \text{poly tail},$$

where  $\partial_l [k(\mathbf{x}, \mathbf{x}_j)]$  means: differentiate the function  $\mathbf{x} \mapsto k(\mathbf{x}, \mathbf{x}_j)$  with respect to its  $l$ -th argument, i.e. exactly  $\partial k / \partial x_l$  evaluated at  $(\mathbf{x}, \mathbf{x}_j)$  — **never** a derivative with respect to the fixed parameter  $\mathbf{x}_j$ .

**Value-row conditions** ( $s(\mathbf{x}_i) = u_i$ ):

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j), \quad G_{ijl} = \frac{\partial k}{\partial x_l}(\mathbf{x}_i, \mathbf{x}_j).$$

**Derivative-row conditions** ( $(\partial s / \partial x_m)(\mathbf{x}_i) = g_{i,m}$ ): differentiating  $s$  again with respect to  $x_m$ , EVERY term in the ansatz picks up one more  $\partial / \partial x_m$  acting on the **same free variable**  $\mathbf{x}$ :

$$\frac{\partial s}{\partial x_m}(\mathbf{x}_i) = \sum_j w_j \frac{\partial k}{\partial x_m}(\mathbf{x}_i, \mathbf{x}_j) + \sum_j \sum_l v_{j,l} \frac{\partial^2 k}{\partial x_m \partial x_l}(\mathbf{x}_i, \mathbf{x}_j) + \dots$$

So the derivative-derivative block is

$$H_{i,m,j,l} = \frac{\partial^2 k}{\partial x_m \partial x_l}(\mathbf{x}_i, \mathbf{x}_j) = \text{kernel\_hessian\_x}(\mathbf{x}_i, \mathbf{x}_j)[m, l],$$

the **ordinary  $\mathbf{x}$ -Hessian** of  $k(\cdot, \mathbf{x}_j)$  evaluated at  $\mathbf{x}_i$  — **no sign flip, no derivative with respect to the centre**.

**The bug and its fix.** An earlier implementation instead computed  $H_{i,m,j,l} = -\partial^2 k / \partial x_m \partial x_l$  — i.e. it mixed a derivative-w.r.t.-evaluation-point with a derivative-w.r.t.-centre, using the (correct, but **inapplicable here**) radial-kernel identity  $\partial_{\mathbf{x}} k = -\partial_{\mathbf{c}} k$ . That identity holds, but the value-row block  $G$  was built using  $\partial_{\mathbf{x}} k$  (matching the ansatz above), while the derivative- derivative block used  $-\partial_{\mathbf{x}} \partial_{\mathbf{x}} k$  (i.e. implicitly  $\partial_{\mathbf{x}} \partial_{\mathbf{c}} k$ ) — an **internally inconsistent** pair of bases. The system still “solved” (it is generically full rank) but converged to a function satisfying neither the value nor derivative data well: empirically, the value error was  $5\text{--}8 \times$  **worse** than a plain value-only RBF fit at the same  $\sigma$ , and gradient error was  $5\text{--}6 \times$  worse — the opposite of the intended super-convergence.

Removing the erroneous sign flip (using the ordinary  $\mathbf{x}$ -Hessian throughout, matching the  $G$  block’s convention) restored the expected behaviour: the Hermite fit now **beats** the value-only baseline on both metrics, exactly matching Theorem 6.1 below.

### 8.2 Super-Convergence Theorem

**Theorem (Wu 1992; Fasshauer 1996).** The Hermite interpolant  $s_h^H$  fitting both  $u(\mathbf{x}_j)$  and  $(\nabla u)(\mathbf{x}_j)$  satisfies

$$\|\nabla(u - s_h^H)\|_{L^\infty} = O(h^\tau) \|u\|_{\mathcal{H}_k},$$

the **same** rate as the **value** error of a value-only interpolant — one full order better than differentiating a value-only fit, which only achieves  $O(h^{\tau-1})$ .

**Numerical confirmation (Runge function, Matérn-5/2,  $N = 25$ ,  $\sigma = 0.15$ , after the fix):**

| Method          | $\ u - s\ _{L^2}$                        | $\ \nabla(u - s)\ _{L^2}$                |
|-----------------|------------------------------------------|------------------------------------------|
| RBF, value-only | $5.6 \times 10^{-2}$                     | $3.8 \times 10^{-1}$                     |
| Hermite (fixed) | $7.1 \times 10^{-3} \downarrow 8 \times$ | $6.4 \times 10^{-2} \downarrow 6 \times$ |

Table 2: Hermite collocation now outperforms the value-only baseline on both value and gradient accuracy, as theory predicts.

### 8.3 Regularisation: Tikhonov vs SVD Truncation

The assembled Hermite system has condition number routinely  $10^7$ - $10^9$  (the derivative block’s diagonal scales as  $\sim 1/\sigma^2$  relative to the value block’s  $O(1)$  diagonal — an inherent scaling mismatch, not fixable by kernel choice alone). Two remedies were compared:

- **Truncated-SVD least squares** (`numpy.linalg.lstsq` with a `rcond` cutoff): discards singular directions below the cutoff. Found to be **unstable** here — sweeping `rcond` over  $[10^{-3}, 10^{-10}]$  never recovered accurate predictions (best case  $\|u - s\|_{L^2} \sim 1$ , useless), because the derivative constraints occupy exactly the low-singular-value subspace that truncation removes, discarding the gradient information the fit was supposed to use.
- **Tikhonov-regularised normal equations** (used in the fixed code):

$$(A^\top A + \lambda I_{\text{kernel block}}) \mathbf{z} = A^\top \mathbf{b}, \quad \lambda = 10^{-6},$$

regularising **only** the kernel/derivative-weight block (not the polynomial block, which must remain exact for the annihilation condition). This is stable across  $\sigma \in [0.05, 0.3]$  and  $N$  up to a few hundred.

### 8.4 Gradient-Enhanced K-KAN Edges (`kkan/model/gradient_enhanced_layer.py`)

An alternative to solving a (fragile) linear system: give the **neural** K-KAN edge an explicit odd-symmetry derivative basis,

$$\varphi_{ij}(x) = \sum_k [w_{ijk} k(r_k) + v_{ijk} k'(r_k) \text{sign}(x - c_k)] + \dots,$$

and train with the **Sobolev** ( $H^1$ ) loss

$$\mathcal{L}_{H^1} = \frac{1}{N} \sum_j \left( u_{\theta}(\mathbf{x}_j) - y_j \right)^2 + \lambda_{\nabla} \frac{1}{N} \sum_j \left\| \nabla u_{\theta}(\mathbf{x}_j) - \mathbf{g}_j \right\|^2,$$

where  $\nabla u_{\theta}$  is obtained by ordinary `jax.grad` through the **whole** network (chain rule through all layers), not a linear collocation system. Since  $k(r)$  is even and  $k'(r) \text{sign}(\Delta x)$  is odd about each centre, together they span both symmetric and antisymmetric local features — analogous to using both cosine and sine terms in a Fourier basis.

**Empirical result:** this combination outperforms **both** the fixed Hermite collocation and the value-only baseline by roughly an order of magnitude on both metrics (Runge,  $N = 30$ ):

$$\|u - u_{\theta}\|_{L^2} = 1.6 \times 10^{-3}, \quad \|\nabla(u - u_{\theta})\|_{L^2} = 2.3 \times 10^{-2},$$

because gradient descent directly optimises the quantity of interest at every step, rather than solving a single (however well-posed) linear system once.

## 8.5 High-Level API (`kkan/model/joint_training.py`)

`fit_with_gradients(x, u, du=..., d2u=..., ...)` wraps the HermiteKKAN + Sobolev-loss + two-stage (Adam→L-BFGS) pipeline into a single call, returning `(model, params)`; `predict_with_gradients` recovers both  $u(\mathbf{x})$  and  $\nabla u(\mathbf{x})$  via `jax.grad` at arbitrary query points — this is the recommended entry point for joint function+gradient learning in this codebase.

## 9 Mixed-Formulation 4th-Order PDEs (`kkan/solvers/coupled_solver.py`)

### 9.1 Why a Single Scalar Network Fails for 4th-Order PDEs

Cahn-Hilliard ( $u_t = \Delta\mu$ ,  $\mu = u^3 - u - \varepsilon^2\Delta u$ ) and the stream-function form of steady 2D Navier-Stokes ( $\Delta\psi = -\omega$ ,  $\mathbf{u} \cdot \nabla\omega = \nu\Delta\omega$ ,  $\mathbf{u} = (\psi_y, -\psi_x)$ , after eliminating  $\omega = -\Delta\psi$ ) both reduce, for a **single** scalar field, to a PDE requiring **fourth**-order spatial derivatives of the network output. Computing this via nested `jax.hessian(jax.hessian(dot))` is exact and — evaluated once — completes in finite (if slow) time. The problem is **training**: LSPG requires  $\nabla_\theta \mathcal{J}$ , i.e. differentiating the **already 4th-order** residual with respect to network parameters — a graph with 5-6 levels of nested automatic differentiation. Empirically this configuration crashes the XLA compiler (segmentation fault) **regardless of model size or quadrature count** — confirmed down to a  $10 \times 10$ -unit, 2-layer network with 10 quadrature points and 3 training steps. This is a genuine limitation of the JAX/XLA compilation of deeply nested `vmap(hessian(hessian(dot)))`-style graphs, not a hyperparameter issue.

### 9.2 The Mixed-Formulation Fix

The standard remedy in the PINN literature for exactly this class of problem (Wight & Zhao 2020 for Cahn-Hilliard) is to introduce an **auxiliary field** and rewrite the 4th-order equation as a **coupled pair** of 2nd-order equations, each trained with its own scalar K-KAN:

**Cahn-Hilliard, split** — fields  $u_\theta$ ,  $\mu_\theta$ :

$$R_1 = \frac{\partial u_\theta}{\partial t} - \Delta\mu_\theta, \quad R_2 = \mu_\theta - (u_\theta^3 - u_\theta - \varepsilon^2\Delta u_\theta).$$

Joint loss  $\mathcal{J} = \sum_i w_i [R_1(\mathbf{x}_i)^2 + \beta_{\text{couple}} R_2(\mathbf{x}_i)^2] + \text{BC/IC terms on } u_\theta$ . Each residual now needs only **one** `jax.hessian` call — the same computational class as the Poisson/Helmholtz solvers that train reliably.

**Navier-Stokes, split** — fields  $\psi_\theta$  (stream function),  $\omega_\theta$  (vorticity):

$$R_1 = \Delta\psi_\theta + \omega_\theta, \quad R_2 = \mathbf{u}_\theta \cdot \nabla\omega_\theta - \nu\Delta\omega_\theta, \quad \mathbf{u}_\theta = (\partial_y\psi_\theta, -\partial_x\psi_\theta).$$

$R_1$  needs one Laplacian of  $\psi_\theta$ ;  $R_2$  needs one gradient of  $\psi_\theta$  and one Laplacian of  $\omega_\theta$  — again all single-Hessian operations.

Both `CahnHilliardSolver` and `NavierStokesCavitySolver` implement this pattern: two independent **KKAN** instances, a joint LSPG loss summed over both residuals (with a coupling weight  $\beta_{\text{couple}}$  balancing the two equations' typical magnitudes), and standard Adam→L-BFGS training — verified to train without compiler crashes at practical model sizes (20-24 hidden units per layer, 30-60 quadrature points).

### 9.3 Boundary Conditions for the Cavity Flow Example

The lid-driven cavity benchmark on  $[0, 1]^2$  uses  $\psi = 0$  on all four walls (no-penetration streamline condition — the domain boundary is itself a streamline). The moving-lid tangential condition  $\partial\psi/\partial y = 1$  on  $y = 1$  is **not** separately enforced in the minimal example provided (a coarse demonstration of the formulation, not a converged high-Reynolds benchmark); a production solver would add this as an additional soft penalty term using the `normal_deriv` / `grad_u`-based operators already available in `kkan/utils/diff_ops.py`.

## 10 Summary Table: Module -> Mathematics Map

| Module                                   | Core mathematics                                                                              |
|------------------------------------------|-----------------------------------------------------------------------------------------------|
| kernels/families.py                      | Positive-definite radial kernels; polynomial augmentation for unisolvence                     |
| kernels/derivatives.py                   | Analytic $k'$ , $k''$ , safe gradient/Hessian/Laplacian at $r = 0$                            |
| model/kan_layer.py                       | Adaptive (Gauss/Chebyshev) grids; MLS partition-of-unity; Hard-Concrete $L_0$ pruning         |
| model/<br>gradient_enhanced_layer.py     | Odd-symmetry derivative-basis edges; Sobolev ( $H^1$ ) loss; native-space (RKHS) regulariser  |
| model/joint_training.py                  | High-level fit/predict API for joint value+gradient learning                                  |
| solvers/variational_solver.py            | Deep Ritz energy minimisation; LSPG quadrature-weighted residual; adaptive collocation; XPINN |
| solvers/meshless_solver.py               | Kansa/Galerkin direct collocation; RKPM/MLS interpolant; backward-Euler MOL                   |
| solvers/coupled_solver.py                | Mixed-formulation split of 4th-order PDEs into coupled 2nd-order pairs                        |
| approximation/<br>hermite_collocation.py | Hermite-Birkhoff value+gradient interpolation; Tikhonov-stabilised solve                      |
| approximation/<br>quasi_interpolation.py | Shepard/Modified-Shepard/Gaussian quasi-interpolants; Strang-Fix reproduction order           |

Table 3: Every source module and the mathematics it implements.

## 11 References

1. Wendland, H. (2004). **Scattered Data Approximation**. Cambridge University Press.
2. Wendland, H. (1995). Piecewise polynomial, positive definite and compactly supported radial functions of minimal degree. **Adv. Comput. Math.** 4(1), 389–396.
3. Wu, Z. (1995). Compactly supported positive definite radial functions. **Adv. Comput. Math.** 4(1), 283–292.
4. Wu, Z. (1992). Hermite-Birkhoff interpolation of scattered data by radial basis functions. **Approx. Theory Appl.** 8(2), 1–11.
5. Fasshauer, G. E. (1996). Solving partial differential equations by collocation with radial basis functions. *Proc. Chamonix*.
6. Schaback, R. (1995). Error estimates and condition numbers for radial basis function interpolation. **Adv. Comput. Math.** 3(3), 251–264.
7. Kansa, E. J. (1990). Multiquadrics — a scattered data approximation scheme. **Comput. Math. Appl.** 19(8-9), 147–161.
8. Liu, W. K., Jun, S., & Zhang, Y. F. (1995). Reproducing kernel particle methods. **Int. J. Numer. Methods Fluids** 20(8-9), 1081–1106.
9. Louizos, C., Welling, M., & Kingma, D. P. (2018). Learning sparse neural networks through  $L_0$  regularization. **ICLR**.
10. Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks. **J. Comput. Phys.** 378, 686–707.
11. Jagtap, A. D., & Karniadakis, G. E. (2020). Extended PINNs (XPINNs). **Commun. Comput. Phys.** 28(5), 2002–2041.
12. Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). DeepXDE. **SIAM Review** 63(1), 208–228.

13. Wight, C. L., & Zhao, J. (2020). Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks. **Commun. Comput. Phys.** 29(3), 930-954.
14. Liu, Z., et al. (2024). KAN: Kolmogorov-Arnold Networks. **arXiv:2404.19756**.
15. Kolmogorov, A. N. (1957). On the representation of continuous functions of many variables. **Dokl. Akad. Nauk SSSR** 114, 953–956.